Unit-1 and Unit-2

Introduction and Basics of SQL

Summary: Intro

→ SQL abbreviated, known as structural query language

→ storing, access and manipulation all such actions are possible using SQL

→ Usually commercial and bigdata companies use SQL for effective data efficacy for business success.

→ For example a small scale food truck business can use their sales and customer data for running daily operations and scaling the scope of business, customer experience improvement e.t.c.

→ Basically not just lumps of data but structured data that serves a specific purpose which benefits the business.

→ There are various types of data bases for learning purposes we deal with mostly widely used and popular relational DB SQL server.

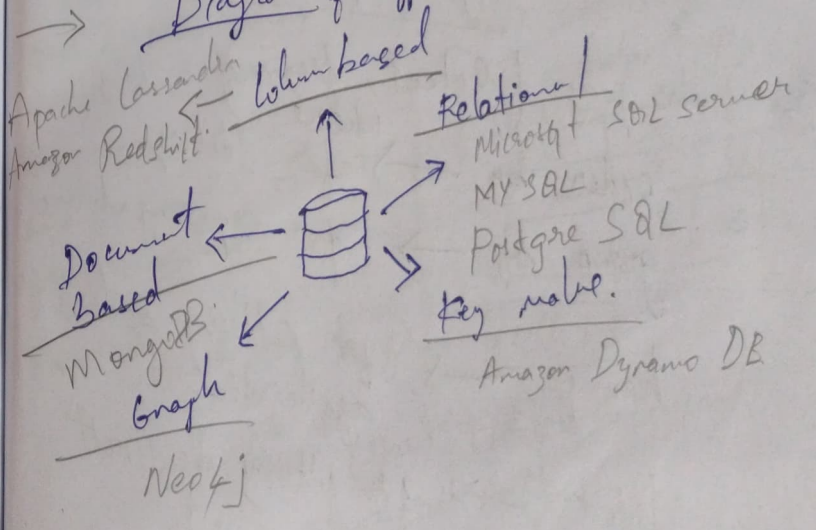
→ like any other service, when trying to operate on data we need to communicate with the database. This happens via a database management system. We have several option.

SSMS

SQL SERVER MANAGEMENT SYS

→ The syntax of SSMS is very simple, user friendly and easy to learn.

→ Diagram of different DB



Summary.

Basics of SQL → for Relational SQL Server (Express) SSSMS-22

→ Select statement for navigating with table data.

table1 Ex:- Select * from table1

id	x	y
1	1	a
2	2	b
3	3	c

this syntax is top down and left right.

the execution order differs.

→ For filtering data we have where statement → based on colnames

Ex: Select * from table1 where x < 2

id	x	y
1	1	a

In similar way, (Select, from, where)

order by → for sorting

group by → for aggregating

having → for filtering after aggregations

distinct → for filtering distinct only.

→ while writing conditions we use operators <, >, !=, :=

!= → not equal to.

e.t.c to let the system know the ranging of condition.

Example Select * from table1 where x != 1

→ this statement gives output as follows

id	x	y
2	2	b
3	3	c

→ we can choose our custom view of output by choosing only columns the we want in the output after executing.

Ex:- Select x, y from table1

→ So, * (star) symbol is depart that outputs all table.

→ the above way x y is custom output Ex:-

x	y
1	a
2	b
3	c

Data definition language.

Unit-3

DDL

→ Usually used when we start creating schema of table ourselves and add or omit columns. Since we are not inputting, here Select does not come into play.

Example

Create table tablename (.....)

table → Syntax

tablename → actual tablename

(.....) → columns (or) schema.

1. create

Example.

```
create table persons (
    id int not null,
    name varchar(50) notnull,
    B.date date,
    phone varchar not null,
    constraint pk_persons primary key (id).
```

(2) Alter. ⟶ using add & drop.

to add new columns or drop columns from existing tables.

Example alter table X add col1 not null,

(3) Drop. adds a col1 to table X and does not accept empty values ⟵ drop column col2

Drop works same

Example: Drop column Colname

Drop table tablename.

Unit-4

Data Manipulation language

DML -

- Insert → new rows to table.

→ we can insert values into new rows.

manually we can do it

Example :-

Insert into tablename (columns)
values
(values to be replaced in columns
., ., ...)
↓
if multiple rows then
(., ., ., ...),
(., ., ., ...)

Insert using Select

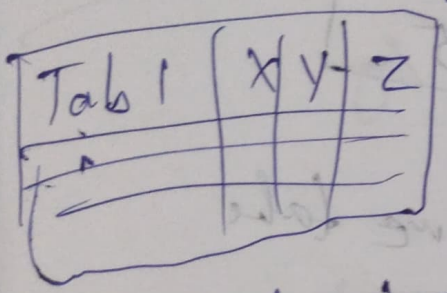
Ex:- insert into ~~table~~ intacted table (columns names)
Select ~~values~~
(~~values~~ in above mentioned columns

Ex:-

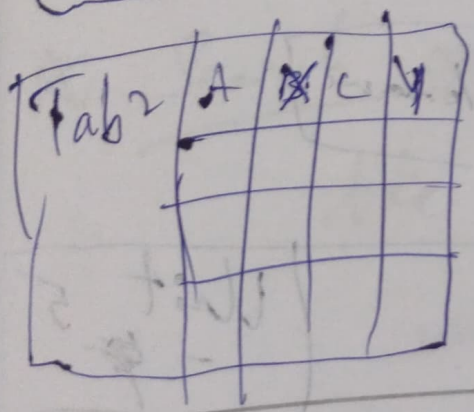
insert into [Intented table] (column names) A, X, C, Y, Z.

↓ Tab 2

select value for A, X, value for C, Y; from Tab 1



→ we are putting data from Tab 1 to Tab 2



Leet code : Combine 2 Tables

② Update → to update values in columns in a table

Ex: Update [tablename]

Set col1 = value1,

:

where (.....);

→ delete → deleting rows of table with conditions using where.

Eg: delete tablename where id > 5

(or) entire table

delete tablename

Unit 5

→ Operators

Comparision operators	Logical operators
<	AND
>	OR
<=	NOT
>=	
!= <>	

Range operators

between and or

membership operators

or (additional rules condition or list using in)

* Search operators

like.

using (% and _)

where col1 = '%.m'

If col1 has values

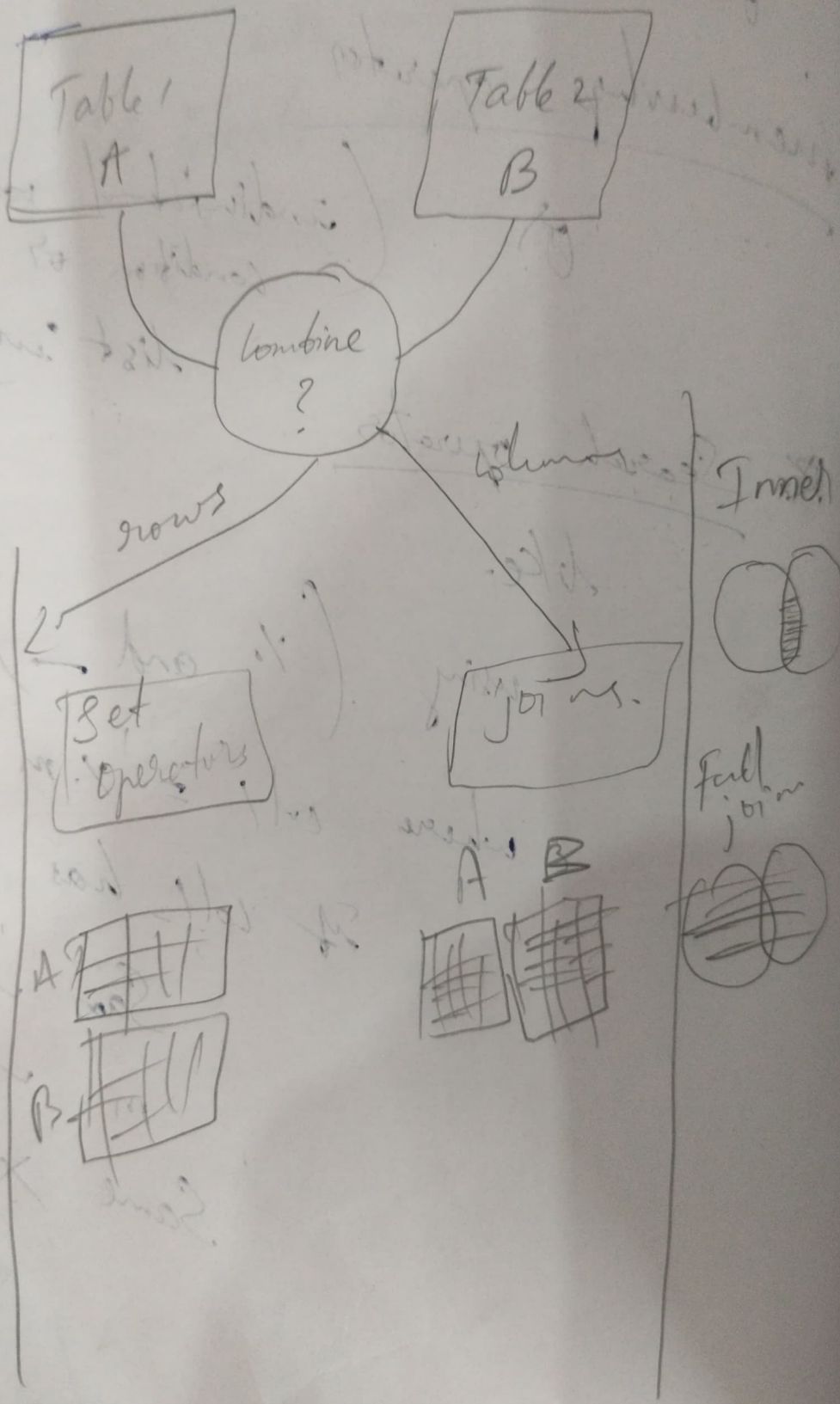
Sam ✓

m ✓

Same ✗

SQL Joins

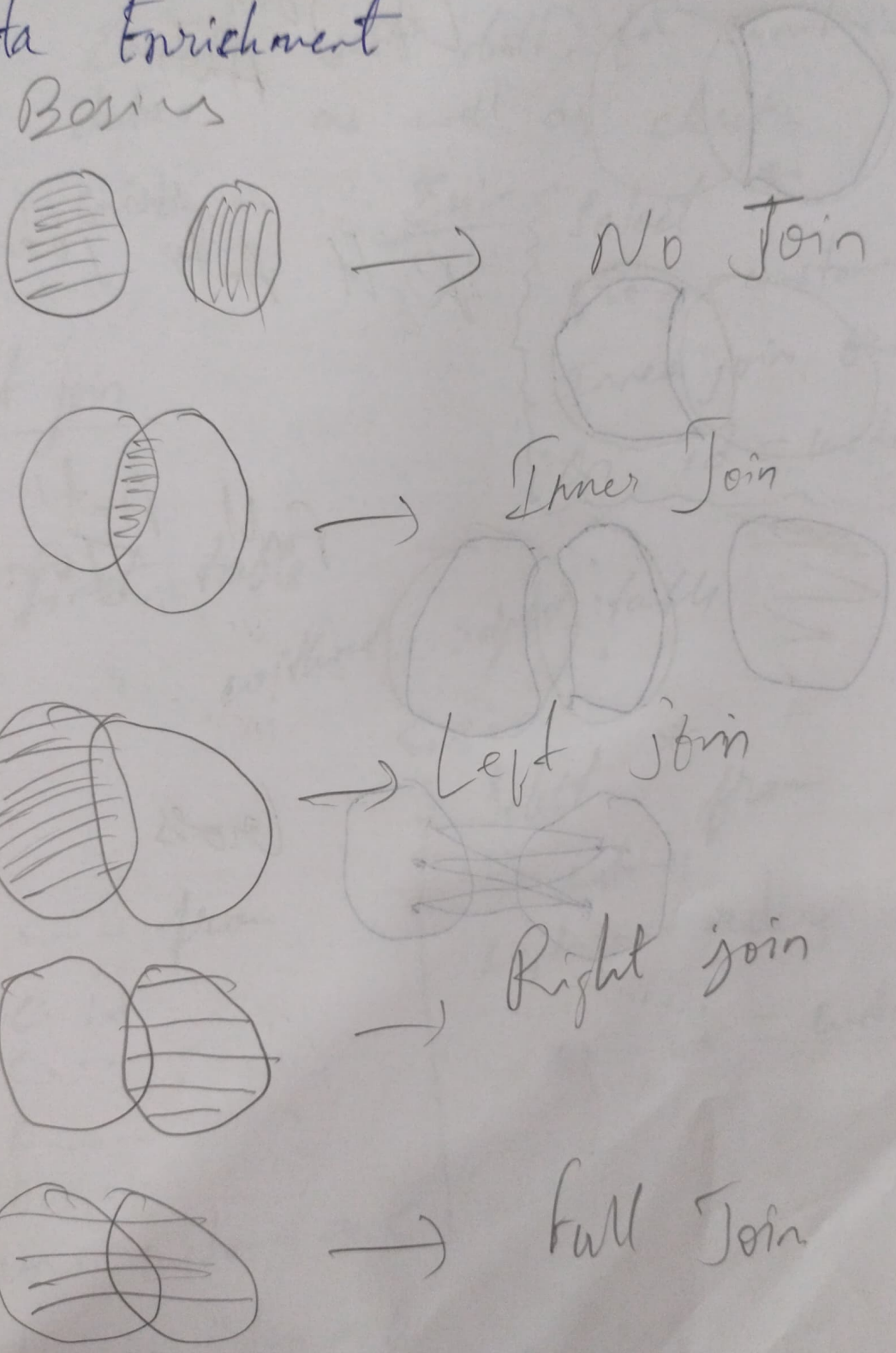
Unit 6



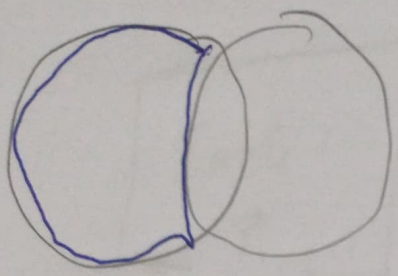
Recombine data

Data Enrichment

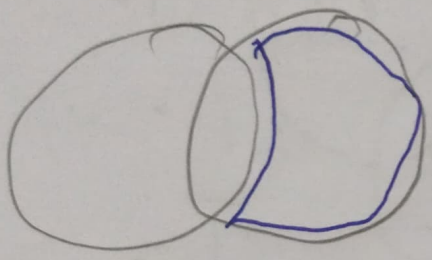
Basics



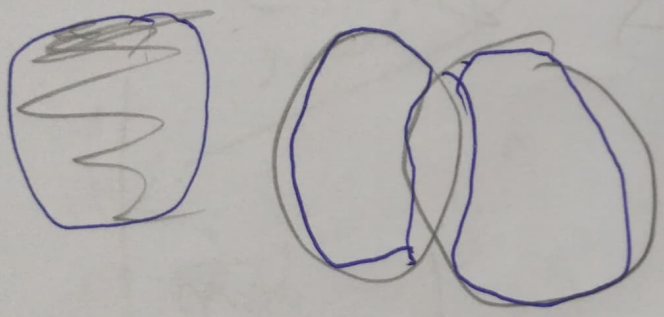
Advanced joins



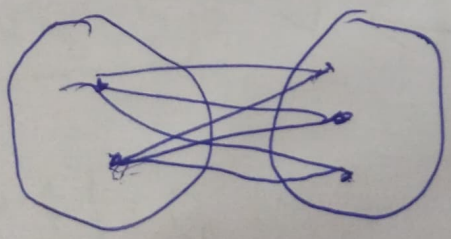
Left Join
Anti



Right Anti Join



Full Anti
Join



Cross

Inner join

Can be used both for combining datas as well as checking existing.

Eg:-
```
Select *
from customer
Inner join orders
on id = customer_id.
```

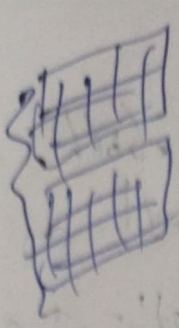
Left join

first table
without right table

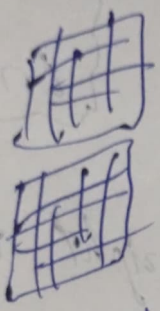
Eg:-
```
Select *
from
customers
Left join orders
on id = customer_id
```

```
S. ... from
    C. id
    C. ... ;
    O. ...
    O. ...
from customers as C
Left join orders as O
on C.id = O.customerid
```

Set Operators

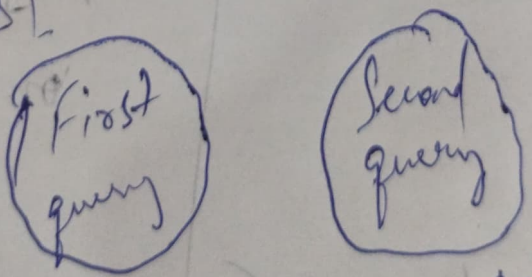
Union →  rows adding.

Note:- Duplicates ignored

Union All → 

Note:- Duplicates included.

Except (Minus)



returns unique in one table that are not in other table.

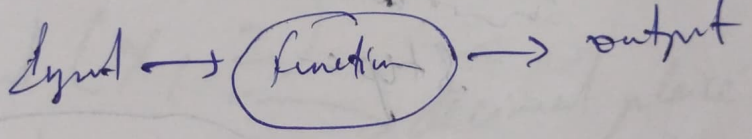
Intersect

→ common rows between two tables.

Row level functions — Unit-7

→ What is a function.

perform action which takes input and gives output

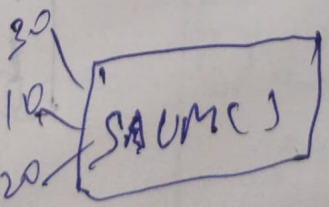


Types

Single row functions

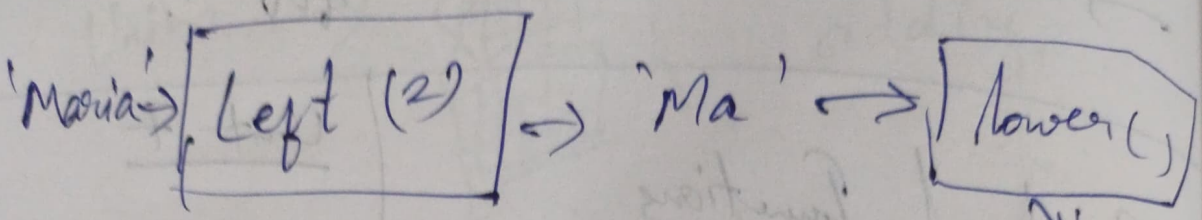
'MARIA' → [lower()] → 'maria'

multiple row functions



Nested funct

Nested functions



len (lower (left ('Maria', 2)))

Step 1

Step 2

Step 3

SQL function

Single row

multiple row

String function

Numeric functions

Date & time

Null functions

Aggregate functions

Window functions

→ String functions.

Manipulate	Calculation	String extraction
concat	len	left
upper		right
lower		Substring
trim		
replace		

→ Numeric functions

Round (X, 2) → decimal place
Round (X, 1) → decimal place
to round off.

abs ( ) → absolute.

→ Date & Time

Date ( )

Time ( )

get date

eve. can divide date to part

(1) 2026 Year
05 month
-1 day

(2) We can add & find diff to dates.

(3) change formats

(4) check for validation mostly
T F

Date & Time functions

- Part Extraction
- Format & Casting
- Calculations

DAY()
month()
year()
datepart() } int

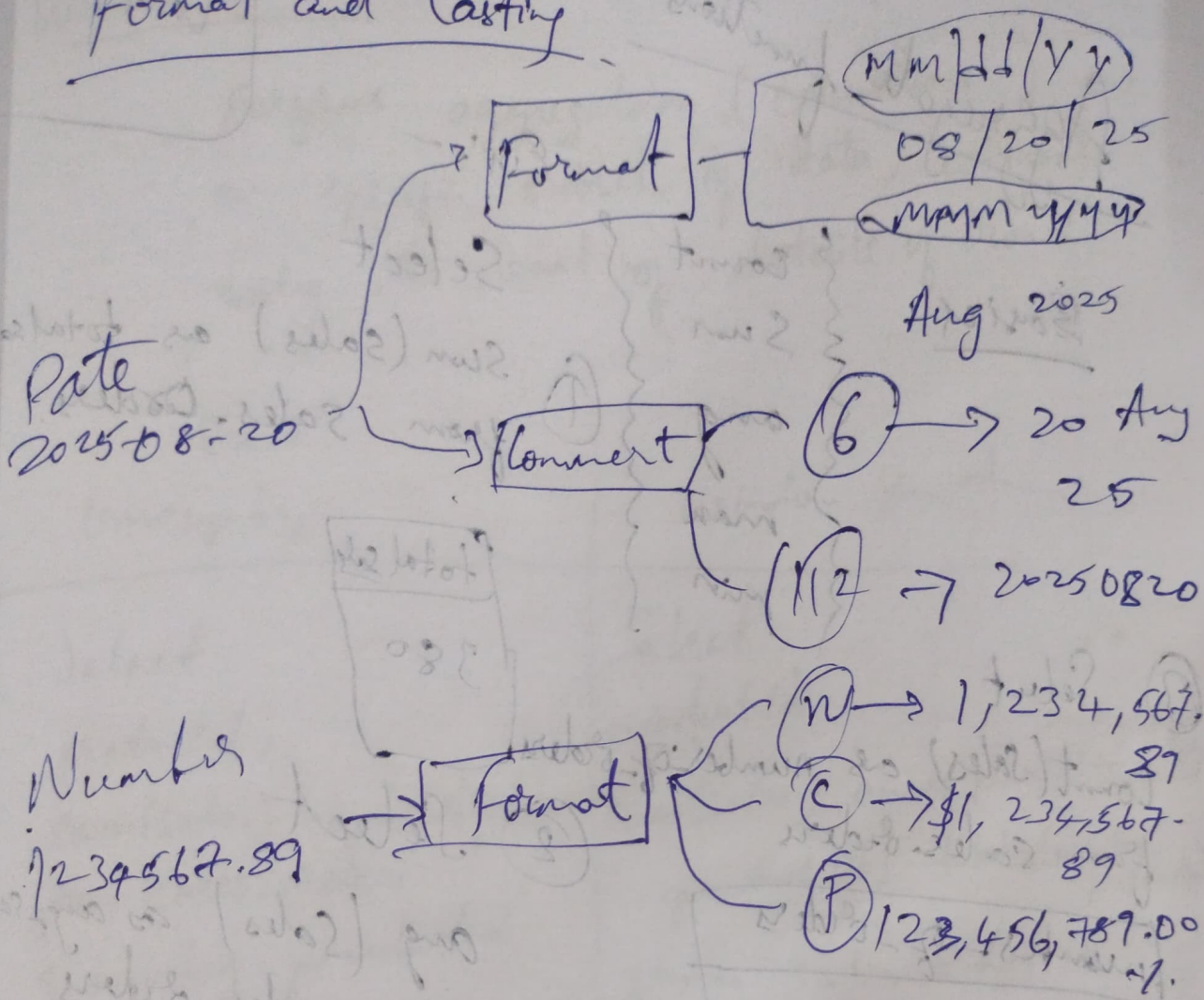
format
convert
cast

datename() } string

datetrunc() } date-time

EOmonth() } date

Format and Casting



→ formatting is changing how value looks like.

→ Casting.

Guest → row level functions

please refer SQL unit 7 file

Unit-8

Aggregate functions

Ex:-

Basics:

- count
- sum
- avg
- max
- min

① Select
sum(sales) as totalsales
from sales.Orders

totalsales
380

② Select
count(sales) as number_of_orders
from sales.orders

number_of_orders
10

② Select
avg(sales) as avg_sales
from sales.orders

avg_sales
38

Source

sales.orders

order id	sales

Window functions

Performs aggregation (calculations) on a specific subset of data, without losing the level of details of rows.

Ex:-

Group by	Window function

```
Select
productid,
sum(sales) as totalsales
from sales.orders
Group by productid
```

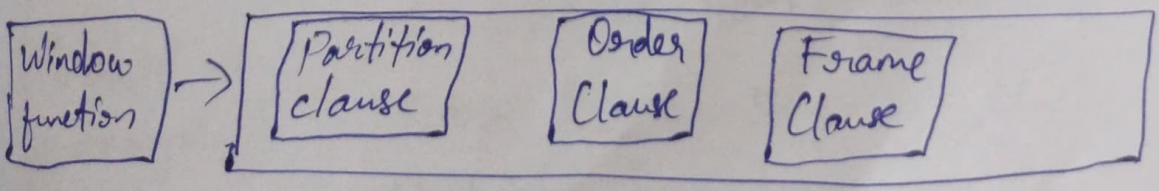
productid	totalsales

```
Select
productid
sum(sales) over(partition by
    productid) totalsales_by_productid
from sales.orders
```

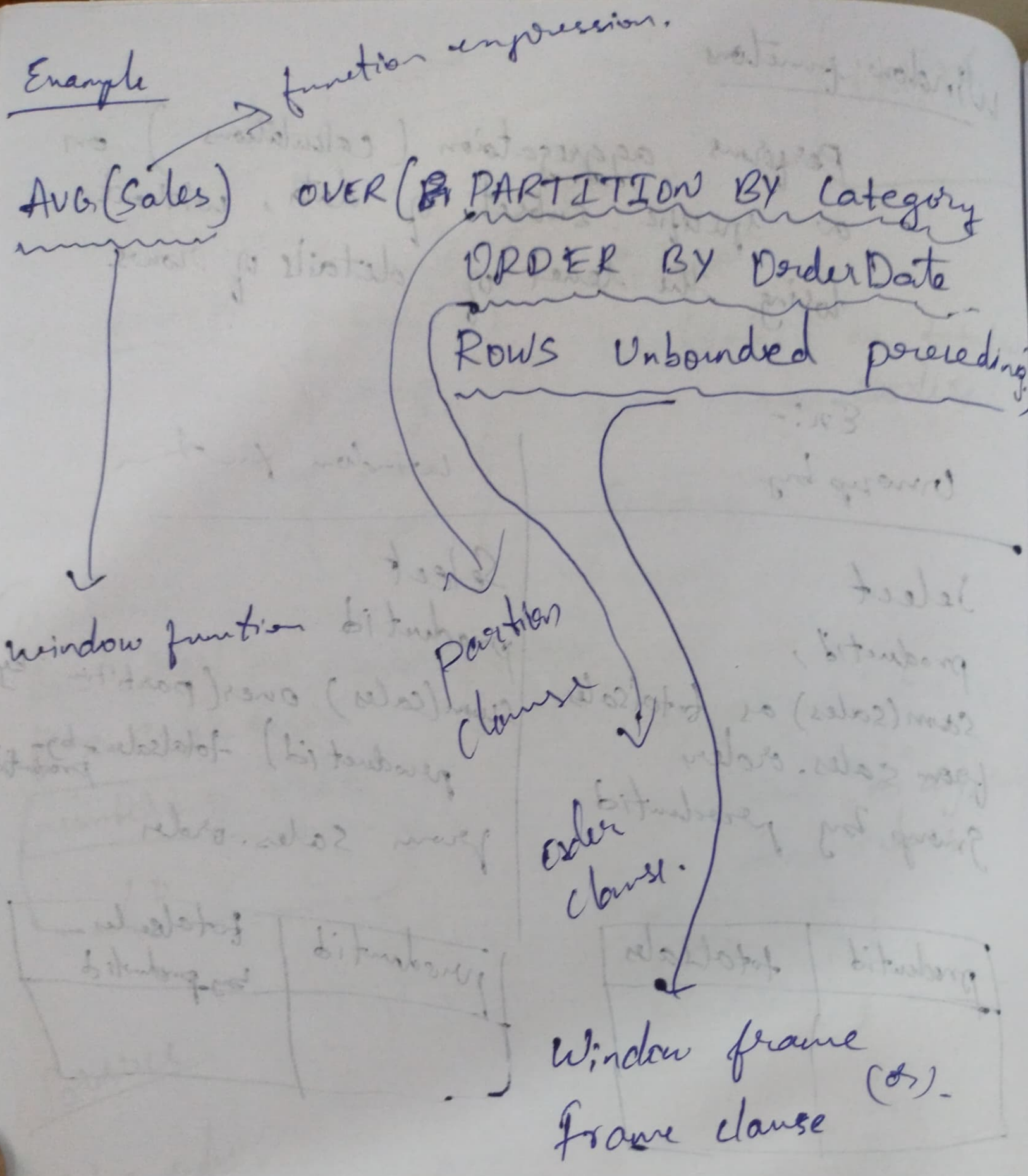
productid	totalsales_by_productid

Syntax.

over() clause



Example


function expression.
AVG(Sales) OVER(PARTITION BY Category
ORDER BY OrderDate
ROWS Unbounded preceding
window function
partition clause
order clause.
Window frame (or).
frame clause

General window functions

Aggregate functions

- count ( )
- sum ( )
- Avg ( )
- Max ( )
- Min ( )

Rank Functions

- ~~Row ( )~~
- Row_number ( )
- Rank ( )
- Dense_Rank ( )
- Cume_dist ( )
- Percent_rank ( )
- Ntile

Value (Analytical) functions

- LEAD (exp, offset, default)
- LAG (")
- firstvalue ( )
- Lastvalue ( )

Example

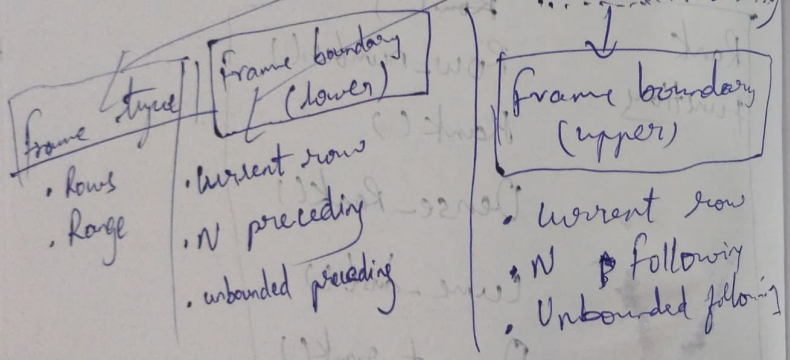
RANK() over(partition by month order by Sales Des

Window frame (or) frame clause

usually used to deal with subset of data.

Example:

AVG(Sales) over(partition by Category Order by Orderdate Rows between current row and unbounded following



Note

→ We cannot use frame clause without order by clause.

→ lower < upper, so should come before.

Rules for Window Functions (what's allowed, what's not).

- window functions cannot be used for filters.
- window function can be used only in select and order by clauses.
- Nested window functions is not allowed.
- The SQL executes window function after where clause.

Advanced SQL functions

- **Data Warehouse**

A special database that collects & integrates data from different sources to enable analytics & support decision-making.

- Different users write their queries and they are independent. Usually there is redundant queries and some users it is time consuming.

Challenges

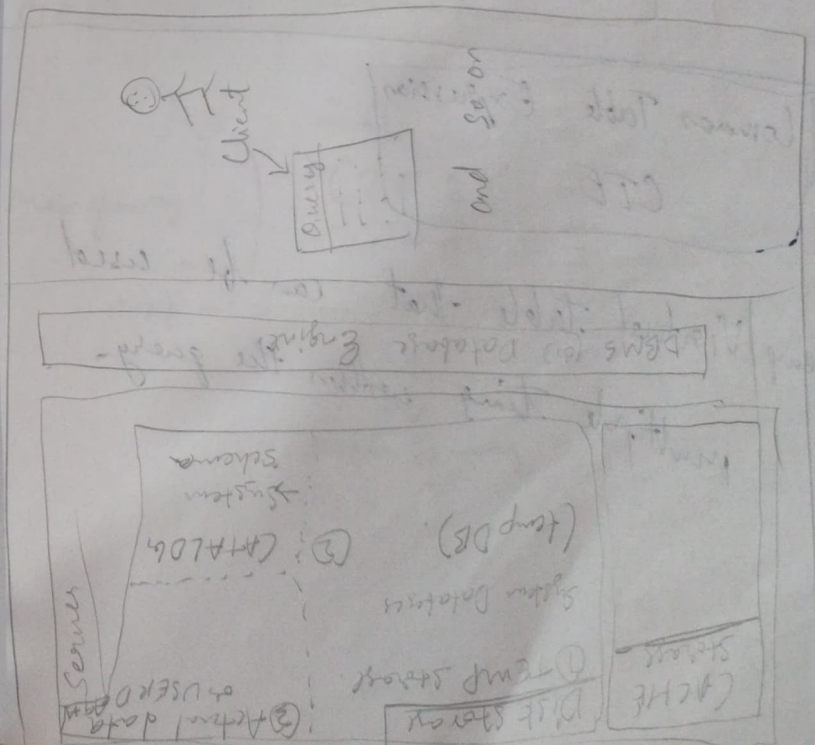
- Redundancy
- Performance issues
- Complexity
- Hard to maintain
- DB Stress
- Security.

To address this we have

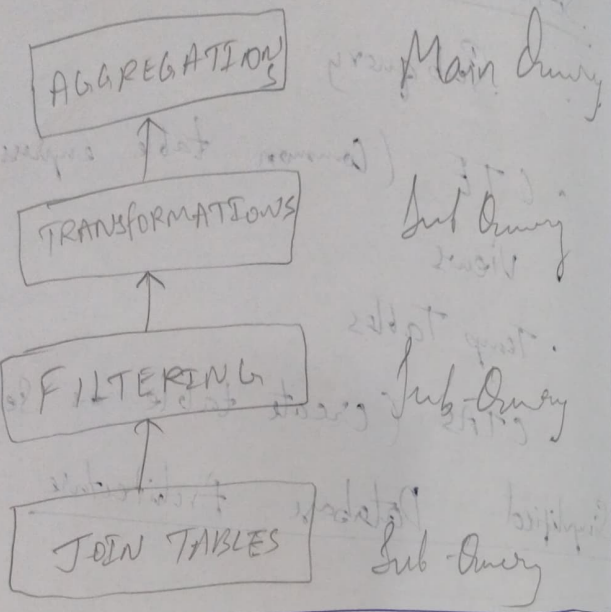
Solutions

- Subquery
- CTE (Common table expressions)
- Views
- Temp Tables
- CTAS (create table as Select)

Simplified Database Architecture



→ SUB-QUERIES

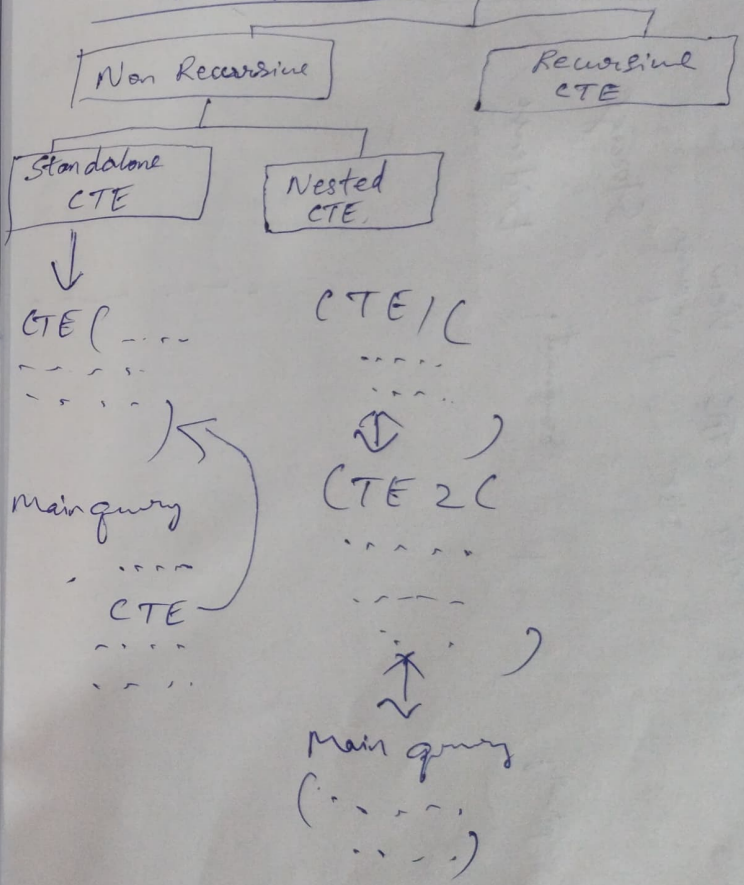


Common Table Expression
CTE

Temp | Virtual table that can be used multiple time within the query.

→ CTE is efficient, reusable and reduces lengthy and complex query to simplified query, and simple.

→ Types of CTE



AI & coding.

chatbox

- Individual experience, no coding view.

Co pilot.

- Dynamic
- Both coding and chatbox simultaneously
- Debugging, checking correctness e.t.c